

Presentación Final: "JASPI" pastillero inteligente.

Samuel David Negrete Lancheros - *Ingeniería Mecatrónica (s - 6)* / 1023163197

Jorge Alejandro Torres Gutierrez - *Ingeniería Mecatrónica (s - 6)* / 1053442807

Alessandra Quintero Rois - *Ingeniería Mecatrónica (s - 6)* / 1023302980

Johan Sebastián Eslava Garzón - Docente

Facultad de Ingeniería, Universidad Nacional de Colombia

24 de julio de 2025

Índice

I	Introducción	I
II	Arquitectura Implementada	II
II-A	Arquitectura Digital	II
II-A1	Arduino	II
II-A2	Protocolo de Comunicación	III
II-B	Arquitectura Física	IX
II-B1	Prototipado	IX
II-B2	Parametros de Diseño	XI
II-B3	Impresión	XI
III	Pruebas	XI
IV	Reporte de IA	XII
V	Conclusiones	XII
	Referencias	XIII

I. Introducción

El cumplimiento adecuado de la administración de medicamentos representa un desafío significativo para la población de adultos mayores. A menudo, la rutina diaria de tomar múltiples pastillas en horarios específicos se ve obstaculizada por diversos factores. En primer lugar, la capacidad de recordar la toma de cada medicamento puede verse comprometida con la edad, especialmente en aquellos individuos que no poseen o no están familiarizados con el uso de dispositivos móviles y sus funciones de alarma. Esta falta de recordatorio oportuno puede llevar a la omisión de dosis o a la toma incorrecta, afectando negativamente su salud y bienestar.

En segundo lugar, la manipulación física de los pastilleros tradicionales puede resultar compleja para algunos adultos mayores. La dificultad para abrir los compartimentos individuales y extraer las pastillas, ya sea por limitaciones en la destreza manual, problemas de visión o condiciones como la artritis, añade una barrera adicional al proceso de medicación. Esta dificultad puede generar frustración, dependencia de terceros o, en el peor de los casos, la imposibilidad de acceder a la medicación prescrita.

La motivación para el desarrollo de este proyecto surge también desde una experiencia personal. La abuela de uno de los integrantes debía tomar una pastilla diaria para controlar su presión arterial. Durante varias semanas olvidó hacerlo, y un coágulo se alojó en su cerebro. En cuestión de meses, su salud se deterioró gravemente. Este hecho marcó profundamente la percepción de la adherencia al tratamiento médico, y evidenció la necesidad urgente de un sistema accesible que ayude a recordar y facilitar la toma de medicamentos.

Si bien se reconoce que una solución tecnológica no garantiza una independencia total, se anticipa que un sistema electrónico digital puede mejorar significativamente esta situación. Al facilitar el recordatorio y la dispensación de las dosis, se espera disminuir la necesidad de supervisión constante por parte de familiares o personas de apoyo.

Por lo tanto, existe una necesidad evidente de desarrollar una solución tecnológica que facilite y asegure la correcta toma de medicamentos en adultos mayores, abordando tanto el desafío del recordatorio como la dificultad en la manipulación de las pastillas. Un sistema digital puede ofrecer una alternativa práctica y accesible para mejorar el seguimiento de los tratamientos, fortalecer la autonomía del usuario y reducir la carga sobre quienes lo asisten.

En este informe se describe el proceso completo de diseño y desarrollo del sistema. Se parte de una justificación conceptual centrada en las necesidades del usuario, seguida por el diseño de la arquitectura de módulos y la estructura física del dispositivo. Se documentan las pruebas realizadas con usuarios reales, se presentan los resultados obtenidos y se discuten sus implicaciones. Finalmente, se exponen las conclusiones generales del trabajo.

II. Arquitectura Implementada

II-A Arquitectura Digital

II-A1. Arduino

El componente principal del sistema es **el módulo RTC (Real Time Clock) DS3231**, por lo que comprender su modo de operación *resulta fundamental*. Inicialmente, se planteó desarrollar el proyecto utilizando una placa **Arduino Uno**, con el objetivo de adaptar el sistema a las necesidades específicas del diseño, principalmente el muestreo de la hora en un elemento visual. A través de la **integración del sensor** y el uso de **librerías de Arduino**, se buscaba facilitar la comprensión del **funcionamiento del RTC**.

De este modo, se decidió construir un “seudoproyecto” con *Arduino*, *permitiendo su montaje y verificación práctica* (visualización de la hora actual). A partir de su correcto funcionamiento, en esta etapa se pensaba posible **analizar la lógica empleada** para luego replicarla y adaptarla al entorno de programación en **FPGA**.

A continuación, se describen las partes fundamentales del código desarrollado en *Arduino*, así como su implementación y el funcionamiento general del sistema¹:

1. **Conexión del modulo:** Como se mencionó anteriormente, el **módulo RTC** *se configura una única vez* con la hora actual mediante el dispositivo al que se conecta. Una vez registrada, esta información se conserva gracias a las configuraciones internas del módulo.

```
1  #include <Wire.h>
2  #include "RTClib.h"
3
4  RTC_DS3231 rtc;
5
6  void setup() {
7    Serial.begin(9600);
8    Wire.begin();
9
10   if (!rtc.begin()) {
11     Serial.println("No se encuentra el RTC");
12     while (1);
13   }
14
15   if (rtc.lostPower()) {
16     Serial.println("RTC sin energia, se ajusta la hora...");
17
18     // Configura la hora (una sola vez)
19     rtc.adjust(DateTime(F(__DATE__), F(__TIME__)));
20   }
21 }
```

Listing 1: Configuración de modulo RTC para Arduino UNO.

El DS3231 cuenta con un *oscilador de cristal* de alta precisión y una batería de respaldo integrada (generalmente una celda tipo CR2032), que le permite mantener el conteo del tiempo incluso cuando el sistema principal está **apagado o desconectado de la alimentación** [1]. Esta combinación de componentes garantiza una medición estable y continua del tiempo, lo que lo convierte en una solución confiable para aplicaciones que requieren una referencia temporal constante.

¹La comunicación se establece a través de los pines A4 (SDA) y A5 (SCL) del Arduino, los cuales están asociados al protocolo de comunicación *I²C*. Esta interfaz es gestionada mediante la librería **RTClib**, que facilita la interacción con el dispositivo. Dicha librería permite acceder de forma directa a objetos **DateTime**, los cuales contienen la información actual de hora y fecha, simplificando así la lectura y manipulación de los datos temporales en el entorno Arduino.

2. **Arreglo de la hora:** Una vez obtenida la hora actual, el valor es separado en tuplas de decenas y unidades para cada uno de los dígitos que se desean mostrar. Para ello, se emplea el recurso matemático del módulo (%) y la división entera (/).

```

1  digits[0] = hour / 10;
2  digits[1] = hour % 10;
3  digits[2] = minute / 10;
4  digits[3] = minute % 10;

```

Listing 2: Artilugio Matemático para capturar de tuplas a un único valor.

3. **Muestreo:** El paso anterior permite preparar los datos para su visualización en un display de 7 segmentos de ánodo común², donde los segmentos se activan poniendo el pin a nivel bajo (*LOW*). El código implementa una función de refresco que utiliza multiplexado dinámico para encender los dígitos uno a uno con una breve demora, simulando que todos están activos al mismo tiempo.

```

1  void refreshDisplay() {
2      for (uint8_t d = 0; d < 4; d++) {
3          digitalWrite(digPins[d], HIGH); // Activar dígito (ánodo común)
4          setSegments(segMap[digits[d]]); // Mostrar número correspondiente
5          delayMicroseconds(2000);
6          digitalWrite(digPins[d], LOW); // Apagar dígito
7      }
8  }

```

Listing 3: Función para visualizar dinámicamente.

La función **setSegments()** toma un patrón binario que representa el número a mostrar y enciende los pines correspondientes de cada segmento, invirtiendo los valores debido a la lógica de ánodo común.

```

1  void setSegments(uint8_t pattern) {
2      for (uint8_t s = 0; s < 7; s++) {
3          bool bit = (pattern >> s) & 0x01;
4          digitalWrite(segPins[s], bit ? LOW : HIGH); // LOW para encender
5      }
6  }

```

Listing 4: Función para configurar ánodo común.

4. Implementación Física:

Luego de cargar el código y ajustar los elementos físicos, se observó la hora actual en los 7 segmentos.

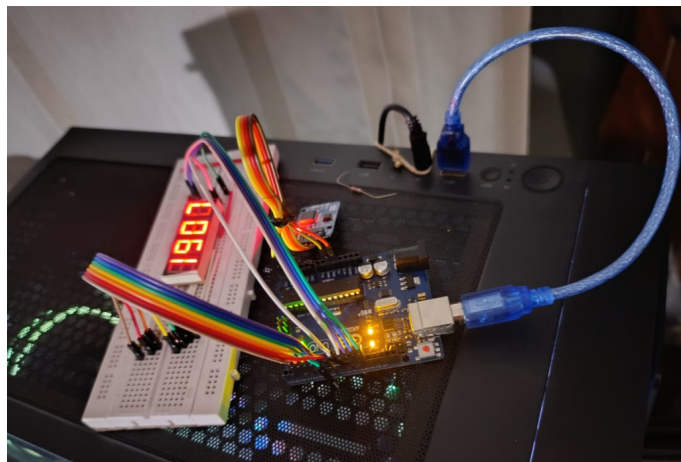


Figura 1: Implementación física de RTC con Arduino UNO en 7 segmentos de 4 dígitos.

II-A2. Protocolo de Comunicación

El módulo comentado al seguir una comunicación I^2C obliga crear el protocolo de comunicación que sirva para relacionar la información con la FPGA. Para poder comprenderlo a un nivel bajo de abstracción fue necesario hacer uso de un analizador lógico, por medio de la interfaz gráfica Pulseview sobre ambos cables de comunicación tras la implementación de la subsección II-A1: véase fig: 2

²El proyecto usó una **LCD** a diferencia de unos 7 segmentos


```

25     end else if (counter == 2*Ns) begin
26         sda_out <= 0;
27         scl <= 1;
28         counter <= counter + 1;
29     end else if (counter == 3*Ns) begin
30         sda_out <= 0;
31         scl <= 0;
32         counter <= counter + 1;
33     end else if (counter == 3*Ns + Delay) begin
34         sda_out <= 1;
35         scl <= 0;
36         counter = 0;
37         state <= Send_Add;
38         st_prev <= Restart;
39     end else counter <= counter + 1;
40 end

```

Listing 5: Ejemplo de la lógica detrás de uno de los estados: (Start)

Adicionalmente este modulo prepara la lectura de la información con su estado de listen, manda una señal que gestiona la activación de **SDA** en su estado de alta impedancia, pasándole el control al *Slave*.

2. **beg_com** Este módulo se encarga de generar la condición de inicio **Start** del protocolo I^2C cada 2 segundos para iniciar la comunicación adicionalmente la señal generada por este modulo va a ser de utilidad en el módulo listen pues sincroniza la captura de la información.
3. **listen** Responsable de recibir los bits desde el RTC por la línea **sda**, sincronizado con **scl**. Interpreta los datos leídos y detecta los bits de ACK/NACK. Implementa una función para invertir los bits recibidos para un correcto guardado de la información.

```

1
2     always @(posedge scl) begin
3         if ((sda_en == 0) & (count != Ndata)) begin
4             data[count] <= sda;
5             count <= count + 1;
6
7         end else if (count == Ndata) begin
8             count <= 0;
9         end
10    end
11
12    function [7:0] reverse_bits8;
13        input [7:0] in;
14        integer i;
15        begin
16            for (i = 0; i < 8; i = i + 1) begin
17                reverse_bits8[i] = in[7 - i];
18            end
19        end
20    endfunction
21
22    always @(negedge beg) begin
23        Acknowledge[2:0] <= data[2:0];
24        Seconds <= reverse_bits8(data[10:3]);
25        Minutes <= reverse_bits8(data[18:11]);
26        Hours <= reverse_bits8(data[26:19]);
27        Day <= reverse_bits8(data[34:20]);
28        Date <= reverse_bits8(data[42:35]);
29        Month <= reverse_bits8(data[50:43]);
30        Year <= reverse_bits8(data[58:51]);
31    end
32

```

Listing 6: Cargue paralelo

4. **lccdin_mod** Se encarga de gestionar el envío de la hora para la representación visual en la **LCD**

A continuación, se detallan los componentes principales del módulo:

```

1
2     localparam IDLE          = 3'b000;
3     localparam STORE_DATA    = 3'b001;
4     localparam CONFIG_CMD1    = 3'b010;
5     localparam WR_STATIC_TEXT_1L = 3'b011;

```

```

6 localparam CONFIG_CMD2 = 3'b100;
7 localparam WR_STATIC_TEXT_2L = 3'b101;

```

Listing 7: Definición de estados

Estos valores representan los distintos estados internos de la máquina de control. Cada uno define una etapa del proceso de configuración o escritura hacia la LCD, incluyendo comandos de inicio, escritura de datos en cada línea, y control del flujo interno.

```

1 //Todas inician en 0
2 wire [7:0] Hours_dec = Hours[7:4] + 8'd48;
3 wire [7:0] Hours_uni = Hours[3:0] + 8'd48;
4 wire [7:0] Min_dec = Minutes[7:4] + 8'd48;
5 wire [7:0] Min_uni = Minutes[3:0] + 8'd48;
6 wire [7:0] Sec_dec = Seconds[7:4] + 8'd48;
7 wire [7:0] Sec_uni = Seconds[3:0] + 8'd48;
8

```

Listing 8: Conversión de BCD a ASCII

Esta sección realiza la conversión de los datos de hora codificados en BCD (típico de un RTC) al formato ASCII requerido por la pantalla LCD. Se aprovecha el hecho de que los números del 0 al 9 en ASCII van del 48 al 57, por lo que simplemente se suman 48 a cada dígito.

```

1 // Siempre que hay un flanco guarda la informacion en text
2 always @(posedge clk) begin
3     text[0] <= 8'h48;
4     text[1] <= 8'h6F;
5     text[2] <= 8'h72;
6     text[3] <= 8'h61;
7     text[4] <= 8'h20;
8     text[5] <= 8'h20;
9     text[6] <= Hours_dec;
10    text[7] <= Hours_uni;
11    text[8] <= 8'h3A;
12    text[9] <= Min_dec;
13    text[10] <= Min_uni;
14    text[11] <= 8'h3A;
15    text[12] <= Sec_dec;
16    text[13] <= Sec_uni;
17    text[14] <= 8'h20;
18    text[15] <= 8'h20;
19
20
21    text[16] <= 8'h20;
22    text[17] <= 8'h20;
23    text[18] <= 8'h20;
24    text[19] <= 8'h20;
25    text[20] <= 8'h20;
26    text[21] <= 8'h20;
27    text[22] <= 8'h20;
28    text[23] <= 8'h20;
29    text[24] <= 8'h20;
30    text[25] <= 8'h20;
31    text[26] <= 8'h20;
32    text[27] <= 8'h20;
33    text[28] <= 8'h20;
34    text[29] <= 8'h20;
35    text[30] <= 8'h20;
36    text[31] <= 8'h20;
37 end
38
39 // Guardo los estados
40 reg [2:0] fsm_state;
41 reg [2:0] next_state;
42 reg clk_16ms;

```

Listing 9: Buffer de cambio dinamico y memoria de estados

Aquí se construye el buffer que contiene los caracteres a mostrar. La palabra "Hora", seguida por la hora actual en formato HH:MM:SS, se codifica en la primera línea. La segunda línea se mantiene vacía por defecto. El texto se actualiza en cada flanco de reloj positivo. También se definen variables para controlar la máquina de estados.

```

1
2 always @(posedge clk_16ms) begin
3     case (fsm_state)

```

```

4      IDLE: begin
5          command_counter <= 0;
6          data_counter <= 0;
7          input_counter <= 0;
8          rs <= 0;
9          data <= 0;
10         last_sw_data <= 0;
11     end
12
13     CONFIG_CMD1: begin
14         rs <= 0;
15         rw <= 0;
16         data <= config_mem[command_counter];
17         command_counter <= command_counter + 1;
18     end
19
20     WR_STATIC_TEXT_1L: begin
21         rs <= 1;
22         rw <= 0;
23         data <= text[data_counter];
24         data_counter <= data_counter + 1;
25     end
26
27     CONFIG_CMD2: begin
28         rs <= 0;
29         rw <= 0;
30         data <= START_2LINE;
31         data_counter <= 0;
32     end
33
34     WR_STATIC_TEXT_2L: begin
35         rs <= 1;
36         rw <= 0;
37         data <= text[NUM_DATA_PERLINE + data_counter];
38         data_counter <= data_counter + 1;
39     end
40 endcase
41 end

```

Listing 10: Máquina de estados

Esta sección implementa la lógica de control mediante una máquina de estados síncrona. Dependiendo del estado en el que se encuentre, el sistema ejecuta comandos de configuración (modo de línea simple o doble), o escribe datos desde el buffer a la LCD, línea por línea. El registro rs indica si se envía un comando (rs = 0) o un dato (rs = 1), y rw = 0 indica que se está escribiendo en la pantalla.

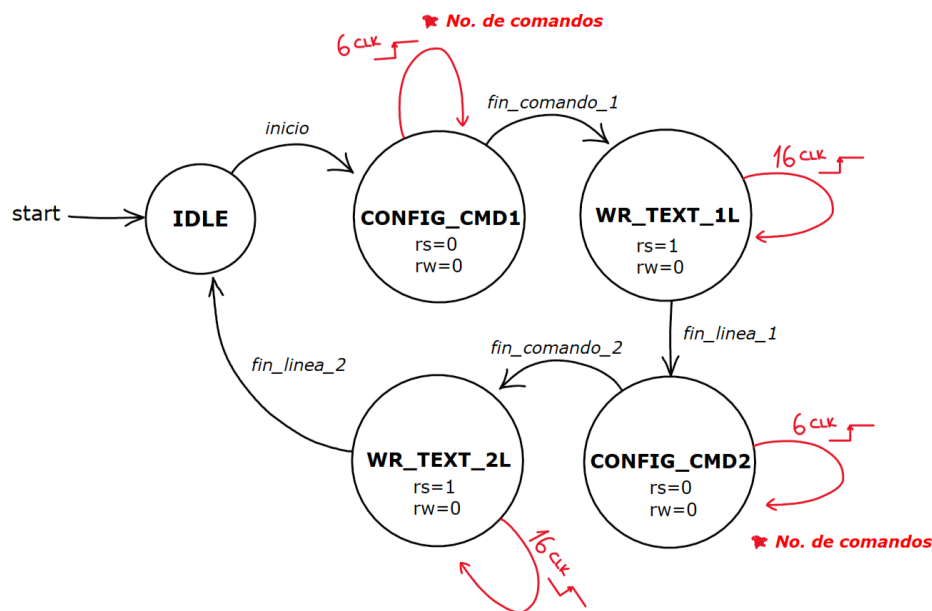


Figura 4: Esquema de la máquina de estados del módulo lcdin_mod

5. **alarm_led** Este módulo gestiona la configuración de horarios de medicación, así como la activación de los indicadores visuales y sonoros.

```

1
2  reg [5:0] intervalo;
3  reg [2:0] led_index;
4  reg buzzer_minute;
5
6  initial begin
7      intervalo <= 6'b000010;
8      last_min_bin <= 0;
9      led_index <= 0;
10 end
11
12 wire [7:0] Minutes_C = (Minutes[7:4] * 8'd10) + Minutes[3:0];
13 wire [7:0] Seconds_C = (Seconds[7:4] * 8'd10) + Seconds[3:0];

```

Listing 11: Definición de condiciones iniciales y entradas en binario

Este fragmento inicial configura los registros del sistema:

- **intervalo**: define cada cuántos minutos debe sonar la alarma.
- **led_index**: índice que permite alternar la activación de LEDs de manera secuencial.
- **buzzer**: indica si la condición del minuto coincidente está activa.
- **Minutes_C** y **Seconds_C** son conversiones de valores BCD a decimal para facilitar las comparaciones aritméticas.

El siguiente bloque contiene la lógica principal que activa la alarma:

```

1
2  always @(posedge clk) begin
3
4      if (Minutes_C % intervalo == 0) begin
5
6          case (led_index)
7              3'd0: leds <= 7'b00000001;
8              3'd1: leds <= 7'b00000010;
9              3'd2: leds <= 7'b00000100;
10             3'd3: leds <= 7'b00001000;
11             3'd4: leds <= 7'b00010000;
12             3'd5: leds <= 7'b00100000;
13             3'd6: leds <= 7'b01000000;
14         endcase
15
16         if (Seconds_C < 6) begin
17             if (
18                 (Seconds_C >= 0 && Seconds_C < 1) ||
19                 (Seconds_C >= 2 && Seconds_C < 3) ||
20                 (Seconds_C >= 4 && Seconds_C < 5)
21             ) begin
22                 buzzer <= 0;
23             end else begin
24                 buzzer <= 1;
25             end
26         end else begin
27             buzzer <= 1;
28         end
29
30         led_index <= (led_index == 3'd6) ? 3'd0 : led_index + 1; // Cambiar los leds en cada
31         coincidencia
32     end else begin
33         buzzer <= 1;
34         leds <= 7'b00000000;
35     end
36 end

```

Listing 12: Ciclo de reloj para alarma y leds.

Este bloque se activa en cada flanco positivo del reloj. Cuando el minuto actual (*Minutes_C*) es múltiplo del valor del *intervalo*, el sistema:

- Enciende un único LED de forma cíclica usando *led_index*.
- Controla el buzzer con un patrón intermitente en los primeros 6 segundos del minuto, emitiendo una señal sonora en los segundos impares (0-1, 2-3, 4-5).
- Reinicia el índice de LEDs al completar el ciclo de 7 LEDs.

emitiendo una señal sonora en los segundos impares (0-1, 2-3, 4-5).

Si el minuto no coincide con el intervalo, se apagan todos los LEDs y el buzzer se desactiva.

6. **top** Es el que instancia todos los módulos anteriores y define las entradas y salidas necesarias para el Pin planer en **Quartus**, gestiona SDA en función de lo que le indique **master** de la siguiente manera.

```
1 assign sda = (sda_en) ? sda_out: 1'bz;
2
3
```

Listing 13: Gestion de SDA

II-B Arquitectura Física

II-B1. Prototipado

Con base en principios de ergonomía y facilidad de impresión, se planteó una estrategia de evaluación centrada en las necesidades de personas adultas mayores. Para ello, se aplicó un cuestionario simple a familiares de diez usuarios potenciales³, en el que se valoraron los siguientes aspectos:

Aspecto evaluado	Promedio (1-10)
Dedos grandes o rígidos	7.4
Sensación temblorosa	6.8
Problemas de memoria	8.2
Reconocimiento de la hora	5.9
Cantidad aproximada de medicamentos diarios	3.6

Tabla I: Evaluación ergonómica percibida en adultos mayores

Con base en estos resultados, se diseñaron diferentes modelos de tipo **mockup** que buscaran resolver la mayoría de las dificultades reportadas. A continuación, se muestra el diseño inicial:

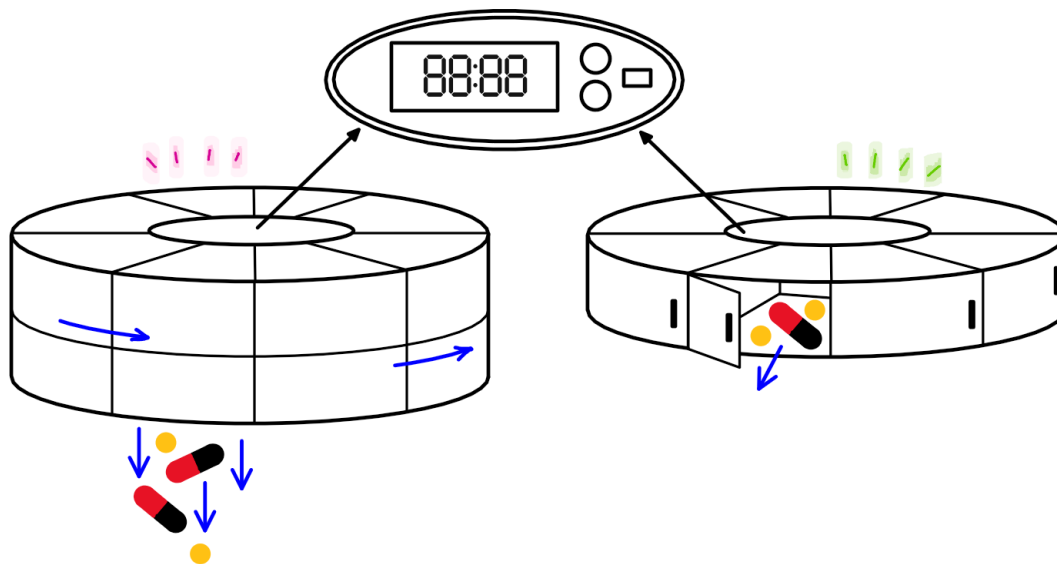


Figura 5: Prototipo inicial diseñado en función de criterios ergonómicos.

Durante esta etapa surgió el reto de diseñar tapas laterales que facilitaran la extracción de la pastilla. Sin embargo, dicha solución resultó poco práctica y potencialmente frágil, debido a la necesidad de múltiples piezas impresas y la instalación de ranuras ajustadas.

Frente a esta limitación, se optó por un enfoque diferente: el diseño de un **deslizador que recorra las distintas casillas del pastillero**, bien sea por día o por horario, permitiendo una dosificación clara y organizada. Este mecanismo ofreció mayor simplicidad mecánica, durabilidad y facilidad de uso para personas con movilidad limitada en las manos.

³Se preguntó a los familiares de 10 personas mayores, considerando sus capacidades físicas y cognitivas.

El diseño final contempla tres componentes principales:

- **Compartimento para la FPGA:** utilizado exclusivamente durante el desarrollo del prototipo. En una futura versión comercial, sería reemplazado por un microcontrolador con un circuito optimizado y de bajo costo.
- **Pastillero central:** el "corazón" del sistema, que contiene las ranuras de almacenamiento y la guía para el deslizador mecánico.
- **Módulo de visualización:** compuesto por un LCD y LEDs de señalización, distribuidos en dos piezas impresas adaptadas al contorno del prototipo.

1. **Deslizador:** Componente móvil encargado del desplazamiento guiado para ubicar el día del compartimento y retirar el contenido (pastillas).

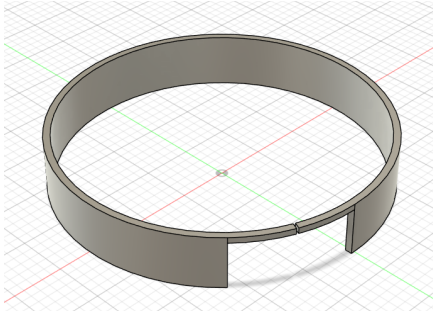


Figura 6: Deslizador: componente móvil del sistema.

2. **Espacio para módulo FPGA:** Alojamiento reservado para el módulo FPGA, con dimensiones adaptadas según los parámetros `largoFPGA` y `anchoFPGA`.

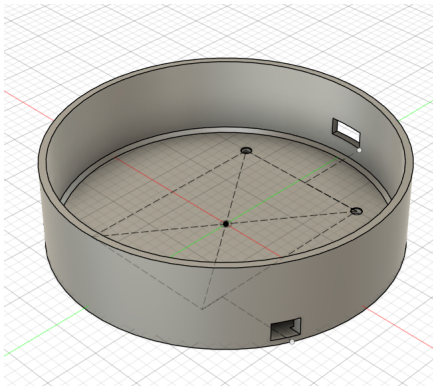


Figura 7: Compartimento para el módulo FPGA.

3. **LCD - Almacenamiento (1):** Primera parte para el montaje del display LCD, muestra una vista del soporte que tiene un agujero para las conexiones previstas.

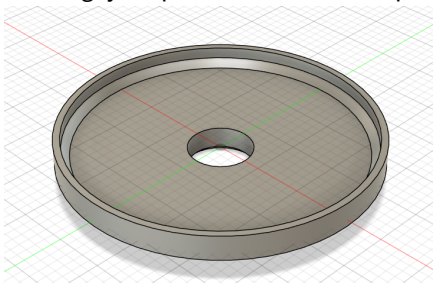


Figura 8: Soporte inferior del display LCD.

4. **LCD - Almacenamiento (2):** Segunda parte tipo "tapa" para el montaje del display LCD. Se observa el espacio

para los LEDs indicadores y el agujero para la pantalla.

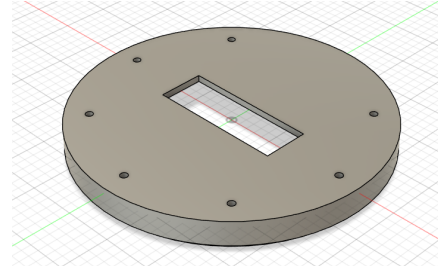


Figura 9: Cubierta superior para el módulo LCD y LEDs.

5. **Pastillero:** Compartimentos que almacenan las pastillas del usuario dividiendo los días. También actúa como puente estructural entre el alojamiento para la LCD y el de la FPGA, permitiendo funcionalidad a través del agujero transversal.

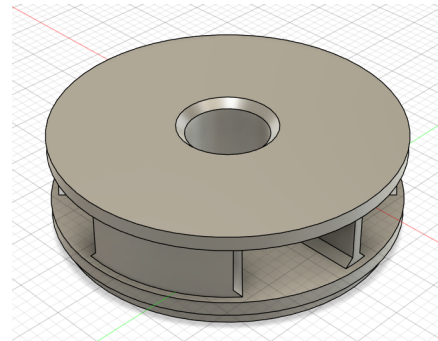


Figura 10: Cuerpo principal del pastillero.

6. **Vista del Ensamble:** Ensamble general del sistema, muestra la integración entre el módulo FPGA, el sistema de visualización y el contenedor principal.

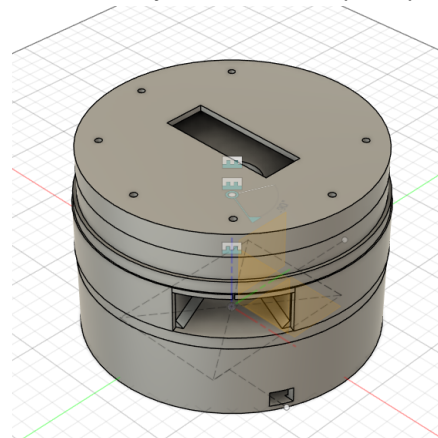


Figura 11: Ensamble completo del sistema.

II-B2. Parametros de Diseño

Los archivos usan parámetros en Fusion para facilitar cambios y mantener uniformidad dimensional. Los siguientes parámetros están definidos en el modelo CAD (Fusion):

Nombre	Unidad	Valor	Descripción
Espesor	mm	7	Espesor general de cada pared
largoFPGA	mm	120	Largo de la FPGA (Valor medido)
anchoFPGA	mm	95	Ancho de la FPGA (Valor medido)
cable1	mm	5	Agujero para el cable 1 de la FPGA
cable2	mm	6	Agujero para el cable 2 de la FPGA
Cilindro	mm	158.673	Diámetro que inscribe la FPGA dentro de sí
largoSSCD	mm	24.45	Largo del módulo LCD (Valor consultado)
anchoSSCD	mm	71.1	Ancho del módulo LCD (Valor consultado)
DimLed	mm	3	Diámetro de led (Valor consultado)
Sep	mm	5	Separación X
CilindroSS	mm	164.614	Diámetro que inscribe el módulo LCD dentro de sí
altSSCD	mm	20	Altura del módulo LCD con pines (Valor medido)

Tabla II: Parámetros físicos de diseño

II-B3. Impresión

Todas las piezas del prototipo fueron fabricadas mediante impresión 3D, utilizando filamento PETG en una impresora Artillery Genius Pro. Este material fue seleccionado por su **resistencia mecánica, durabilidad y estabilidad dimensional** frente a temperaturas moderadas, lo que resulta adecuado para aplicaciones electrónicas y de laboratorio [2] [3].

Durante el diseño, se consideraron cuidadosamente *las tolerancias y dimensiones necesarias para asegurar ajustes funcionales entre componentes*⁴, en especial en las zonas de anclaje y ensamble. Se imprimieron un total aproximado de 500 gramos de material, lo cual equivale a un costo de alrededor de \$60.000 COP (aproximadamente 15 USD al cambio actual).

El diseño propuesto es viable para su réplica, especialmente si se ajustan ciertos aspectos para reducir el consumo de material. Por ejemplo, la pieza inferior del ensamblaje representa cerca de **100 gramos del total**, por lo que optimizar su geometría permitiría disminuir el peso y costo sin comprometer la funcionalidad estructural. Además, si se planea replicar el proyecto para plataformas distintas a la FPGA original, se recomienda redimensionar las cavidades de alojamiento y conexiones para adaptarse a otros controladores o microprocesadores.

A continuación, se presenta una tabla que resume los principales aspectos de viabilidad del diseño:

Aspecto	Valor / Condición	Comentario	Viabilidad
Material	PETG	Buena resistencia mecánica y térmica	Alta
Peso total	500 g	Aproximadamente \$60.000 COP (15 USD)	Moderada
Pieza inferior	100 g	Posibilidad de optimización	Alta
Compatibilidad electrónica	FPGA específica	Redimensionable para otros sistemas	Alta
Método de impresión	FDM - Artillery Genius Pro	Reproducibile con impresoras 3D estándar	Alta
Tolerancias de ensamble	Ajuste funcional logrado	Dimensiones consideradas en diseño	Alta
Replicabilidad	Requiere archivo STL y firmware	Sencillo de compartir y modificar	Alta

Tabla III: Evaluación de viabilidad del diseño impreso

III. Pruebas

Como parte de la validación del sistema, se realizaron pruebas prácticas con un adulto mayor, en este caso el abuelo un abuelo de un integrante. El objetivo fue verificar la usabilidad del prototipo, la claridad de los recordatorios y la facilidad de interacción con el dispositivo.

Durante la prueba se observó que el prototipo resultó comprensible y manipulable sin mayores complicaciones. El sistema de deslizamiento fue intuitivo para ubicar las pastillas, y la señal sonora generada por el buzzer fue lo suficientemente fuerte como para llamar la atención en un entorno doméstico. Asimismo, la visualización de la hora en la pantalla LCD fue clara y legible.

⁴Uno de los integrantes conoce sobre ajustes, su experiencia permitió buenos acabados en la mayoría de las piezas y un acople buen acople a los periféricos.

Estas observaciones sugieren que el sistema puede ser comprendido y operado sin dificultad por adultos mayores, cumpliendo así su propósito principal como herramienta de apoyo para la toma de medicamentos.

A continuación, se muestran fotografías tomadas durante la sesión de prueba en campo:

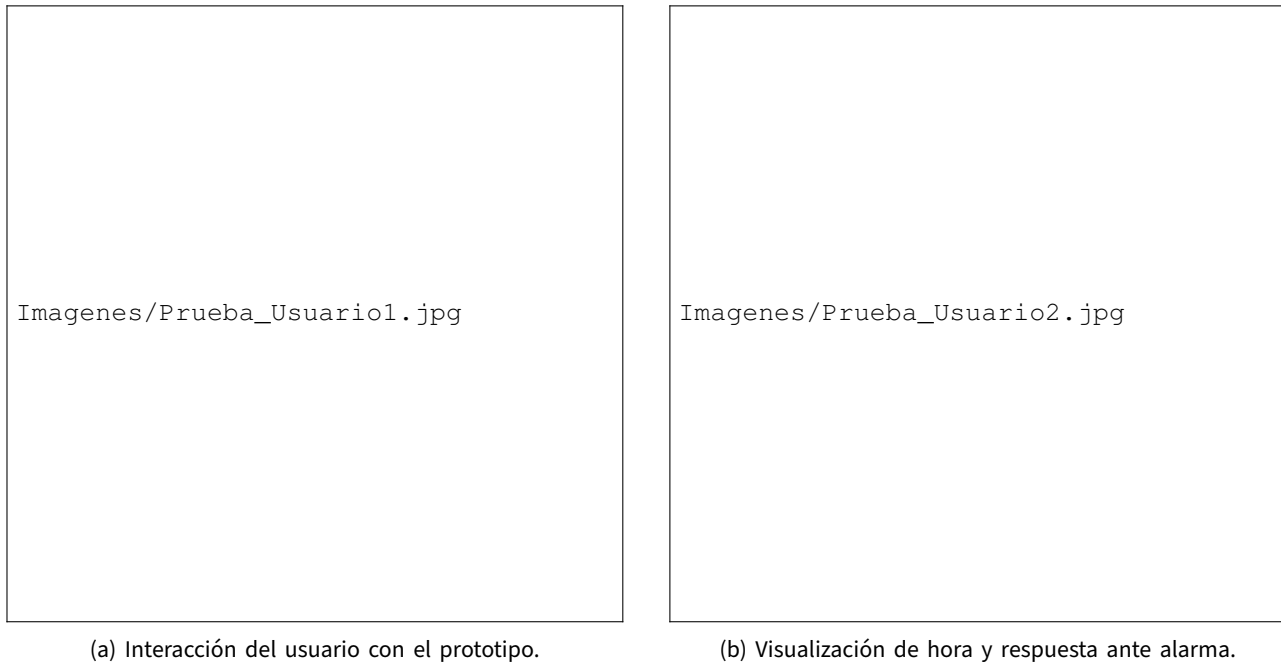


Figura 12: Pruebas con familiar de un integrante.

IV. Reporte de IA

A lo largo del desarrollo del presente proyecto, se contó con apoyo para la recolección, investigación, y tipeado de citas. A continuación, se presenta una lista detallada de los aportes realizados:

- **Recolección de datos y evaluación ergonómica**
 - Diseño de un cuestionario aplicado a familiares de adultos mayores.
 - Investigación de aspectos esenciales para el diseño ergonómico.
- **Revisión de conceptos en digital**
 - Reconocimiento de conceptos particulares en el campo de la electrónica digital.
 - Investigación de conceptos específicos de verilog (wire, rec).
- **Referencias en formato BibTeX**
 - Artículo técnico de Xometry sobre PETG vs PLA.
 - Publicación académica en ResearchGate sobre materiales.
 - Datasheet del módulo DS3231 (Maxim Integrated).

V. Conclusiones

- A pesar de que el diseño propuesto resulta innovador en términos de funcionalidad, ergonomía y usabilidad, *el tamaño de las geometrías necesarias para alojar la FPGA representa una limitación significativa*. La estructura final, **aunque operativa**, presenta dimensiones considerables que la hacen **inviable para una producción a escala** o para un **uso cotidiano por parte de adultos mayores**. Además, el hecho de que la FPGA requiera una alimentación constante compromete aún más su viabilidad como producto de consumo masivo.
- Durante el desarrollo, la implementación de un analizador lógico resultó fundamental para comprender la dinámica del protocolo de comunicación empleado entre el **RTC y la FPGA**. A través de esta herramienta se identificó con claridad el papel esclavo del módulo RTC, el cual transmite información de manera pasiva cuando la **FPGA (maestro) lo solicita**. Esta observación permitió analizar con mayor profundidad el funcionamiento del sistema, validando su comportamiento y abriendo la posibilidad de extrapolar esta lógica a otros protocolos de comunicación serial. En

particular, se logró entender cómo se definen los **modos de operación**, cómo se inicia **la transmisión de datos** y qué condiciones determinan la respuesta del **dispositivo esclavo**.

- En conjunto, este proyecto demostró *la viabilidad técnica de un sistema de apoyo para la administración de medicamentos en adultos mayores*, destacando la importancia de los procesos iterativos de **diseño, prueba y análisis** tanto en el **plano físico** como **lógico**. Si bien se identificaron limitaciones relacionadas con los componentes utilizados, se sientan las bases para una futura implementación con microcontroladores y circuitos optimizados.

Referencias

- [1] M. Integrated, "Ds3231 extremely accurate i2c-integrated rtc with temperature-compensated crystal oscillator," Datasheet, Maxim Integrated, 2015, <https://www.analog.com/media/en/technical-documentation/data-sheets/ds3231.pdf>.
- [2] Xometry. (2023) Petg vs pla: Which 3d printing material should you use? Revisado para elección del filamento. [Online]. Available: <https://www.xometry.com/resources/3d-printing/petg-vs-pla-3d-printing/>
- [3] I. Kovač, J. Slota, S. Lacko, R. Pástor, and V. Fekete, "Comparison of the strength of popular thermoplastic materials used in 3d printing – pla, abs and pet-g," *ResearchGate*, 2024, revisado para elección del filamento. [Online]. Available: https://www.researchgate.net/publication/382404772_Comparison_of_the_strength_of_popular_thermoplastic_materials_used_in_3D_printing_-_PLA_ABS_and_PET-G